

TP Lab 2 — External Events

PIC18F4525 | MPLAB X | Compilateur XC8

Exercice 1 — Timer Switch (Minuterie)

Objectif : Bouton S2 sur RB0, LED D2 sur RA4.

- Appuyer S2 → LED s'allume
- S2 maintenu → LED clignote chaque seconde
- S2 relâché → LED reste allumée 5 secondes puis s'éteint

Calcul du Timer1 pour obtenir 1 seconde

Fosc = 8 MHz → cycle instruction = 0,5 µs

Prescaler 1:8 → 1 tick = 4 µs

Valeur initiale = 65 536 – 250 000 = **15 536 = 0x3CB0**

(250 000 ticks × 4 µs = 1 000 000 µs = 1 seconde ✓)

Code complet — Ex1_Timer_switch.c

```
#include <xc.h>
#pragma config FOSC = INTIO67, WDT = OFF, LVP = OFF, PBADEN = OFF
#define _XTAL_FREQ 8000000

#define S2      PORTBbits.RB0    // Bouton S2 (1=relache, 0=appuye)
#define LED_D2  LATAbits.LATA4   // LED D2

/* Pause de 1 seconde via Timer1 */
void delay_1s(void) {
    T1CON = 0x31;                // Timer1 ON, prescaler 1:8, horloge interne
    TMR1H = 0x3C;                // 0x3CB0 = 15536 -> debordement en 1 seconde
    TMR1L = 0xB0;
    PIR1bits.TMR1IF = 0;        // Effacer le flag de debordement
    while (!PIR1bits.TMR1IF);    // Attendre 1 seconde
    T1CONbits.TMR1ON = 0;       // Arrêter le timer
}

void init_port(void) {
    OSCCON = 0x72;               // Oscillateur interne 8 MHz (IRCF = 111)
    ADCON1 = 0x0F;               // IMPORTANT : toutes les broches en mode NUMERIQUE
    TRISB = 0xFF;                // PORTB en ENTREE (S2 sur RB0)
    TRISA = 0x00;                // PORTA en SORTIE (LED D2 sur RA4)
    LED_D2 = 0;                  // LED eteinte au demarrage
}

void main(void) {
    int count = 0;
    init_port();

    while (1) {
        count = 0;
        while (S2 == 1);         // Attendre l'appui sur S2

        LED_D2 = 1;              // Allumer la LED

        while (1) {
            delay_1s();
            if (S2 == 0) {
                LED_D2 = !LED_D2; // S2 encore appuye : clignoter
            } else {
                count++;           // S2 relache : compter les secondes
                if (count >= 5) break; // 5 secondes ecoules -> sortir
            }
        }
    }
}
```

```
    }  
    LED_D2 = 0;           // Eteindre la LED  
  }  
}
```

POINTS CLES Ex1 : • ADCON1 = 0x0F obligatoire — sans ça les broches sont analogiques et illisibles. • T1CON = 0x31 : TMR1ON=1, prescaler T1CKPS=11 (1:8), horloge interne. • On charge TMR1H:TMR1L = 0x3CB0 pour obtenir exactement 1 seconde de délai.

Exercice 2 — External Interrupt INT0 + Buzzer

Objectif : Bouton S3 sur RB0 (INT0), Buzzer sur RC2.

- count démarre à 26
- Chaque appui sur S3 → interruption INT0 → count--
- Si $\text{count} \geq 7$ → buzzer sonne à 500 Hz
- Après 3 appuis, $\text{count} < 7$ → buzzer s'arrête

Module 1 & 2 — buzzer_on() / buzzer_off()

500 Hz → période = 2 ms → **1 ms à l'état HAUT + 1 ms à l'état BAS**

```
void buzzer_on(void) {
    LATCbits.LATC2 = 1;    // RC2 = 1 pendant 1 ms
    __delay_ms(1);
    LATCbits.LATC2 = 0;    // RC2 = 0 pendant 1 ms
    __delay_ms(1);
    // -> une periode complete = 2 ms = frequence 500 Hz
}

void buzzer_off(void) {
    LATCbits.LATC2 = 0;    // Couper le buzzer immediatement
}
```

Module 3 — init_interrupt()

```
void init_interrupt(void) {
    INTCONbits.INT0ED = 0; // (a) Front DESCENDANT (appui = 1->0)
    INTCONbits.GIE = 1;   // (b) Activer interruptions GLOBALES
    INTCONbits.PEIE = 1;  // (c) Activer interruptions PERIPHERIQUES
    INTCONbits.INT0IE = 1; // (d) Activer INT0 specifiquement
    INTCONbits.INT0IF = 0; // (e) Effacer le flag au depart
}
```

Tableau des bits : INT0ED (INTCON2) : 0=front descendant, 1=front montant GIE (INTCON) : Interrupteur général — si 0, AUCUNE interruption ne marche PEIE (INTCON) : Active les interruptions périphériques INT0IE (INTCON) : Active uniquement INT0 INT0IF (INTCON) : Flag mis à 1 automatiquement quand INT0 se déclenche

Module 4 — ISR (Routine de Service d'Interruption)

```
void __interrupt() ISR(void) {
    INTCONbits.INT0IF = 0; // !! TOUJOURS effacer le flag EN PREMIER !!
    count--;               // Decrementer le compteur
}
// REGLE : si on oublie d'effacer INT0IF -> l'ISR se rappelle en boucle infinie -> programme bloqué
!
```

RÈGLE ABSOLUE : Effacer INT0IF EN PREMIER dans l'ISR. Si on ne l'efface pas, le PIC croit qu'une nouvelle interruption arrive en permanence → rappel infini de l'ISR → programme complètement bloqué.

Module 5 — main()

```
#include <xc.h>
#pragma config FOSC = INTIO67, WDT = OFF, LVP = OFF, PBADEN = OFF
#define _XTAL_FREQ 8000000

volatile int count = 26; // volatile : modifie dans l'ISR ET dans main()

// -- buzzer_on, buzzer_off, init_interrupt, ISR (voir ci-dessus) --

void main(void) {
    OSCCON = 0x72;           // Oscillateur 8 MHz
    ADCON1 = 0x0F;          // Toutes les broches numeriques
    TRISBbits.TRISB0 = 1;   // RB0 = entree (S3)
    TRISCbits.TRISC2 = 0;   // RC2 = sortie (buzzer)

    buzzer_off();           // Buzzer eteint au demarrage
```

```

init_interrupt();          // Activer INT0

while (1) {
    if (count >= 7) {
        buzzer_on();      // Sonne a 500 Hz tant que count >= 7
    } else {
        buzzer_off();     // Buzzer arrete
    }
}
}

```

Pourquoi 'volatile int count' ? Une variable modifiée dans une ISR ET dans main() doit être volatile. Sans ça, le compilateur peut optimiser en mettant count en cache → il ne voit jamais la mise à jour faite par l'ISR.

Exercice 3 — Polling vs Interrupt (Scrutation vs Interruption)

Objectif : comparer deux méthodes pour déclencher le buzzer :

- **Scrutation (Polling) :** lire en permanence RA4 dans la boucle while
- **Interruption :** INT0 sur RB0 déclenche l'ISR automatiquement

Code complet — Ex3_Polling_vs_Interrupt.c

```
#include <xc.h>
#pragma config FOSC = INTIO67, WDT = OFF, LVP = OFF, PBDEN = OFF
#define _XTAL_FREQ 8000000

#define BUZZER LATCbits.LATC2
#define BTN_RA4 PORTAbits.RA4 // Bouton en scrutation sur RA4
#define BTN_RB0 PORTBbits.RB0 // Bouton via interruption sur RB0

void buzzer_on(void) {
    BUZZER = 1; __delay_ms(1);
    BUZZER = 0; __delay_ms(1);
}
void buzzer_off(void) { BUZZER = 0; }

/* ISR : declenchee par RB0 -> reponse immediate */
void __interrupt() ISR(void) {
    INTCON2bits.INT0ED = 0;
    INTCONbits.INT0IF = 0; // Effacer le flag (obligatoire)
    buzzer_on();
}

void init_port(void) {
    OSCCON = 0x72;
    ADCON1 = 0x0F;
    TRISAbits.TRISA4 = 1; // RA4 = entree (scrutation)
    TRISBbits.TRISB0 = 1; // RB0 = entree (interruption)
    TRISCbits.TRISC2 = 0; // RC2 = sortie (buzzer)
    buzzer_off();
    /* Configuration interruption INT0 */
    INTCON2bits.INT0ED = 0; // Front descendant
    INTCONbits.INT0IF = 0; // Effacer le flag au depart
    INTCONbits.INT0IE = 1; // Activer INT0
    INTCONbits.PEIE = 1;
    INTCONbits.GIE = 1;
}

void main(void) {
    init_port();

    while (1) {
        /* --- Scrutation (Polling) sur RA4 --- */
        if (BTN_RA4 == 0) { // Bouton appuye ?
            buzzer_on();
        } else {
            buzzer_off();
        }
        /* L'ISR sur RB0 peut s'intercaler a tout moment -> reponse immediate */
    }
}
```

Tableau de comparaison Polling vs Interruption

Critère	Scrutation (RA4)	Interruption (RB0)
Réactivité	Dépend de la boucle	Immédiate
Peut rater un appui court ?	Oui (si dans delay_ms)	Non — jamais
Charge CPU	Élevée (boucle continue)	Faible

Complexité du code	Simple	Légèrement plus complexe
--------------------	--------	--------------------------

CONCLUSION Ex3 : Si le programme exécute `__delay_ms(1)` au moment où on appuie sur RA4, l'appui est RATÉ (le programme ne lit pas RA4 pendant ce temps). Avec RB0 + interruption, le hardware détecte l'appui INSTANTANÉMENT et l'ISR s'exécute dès la fin de l'instruction en cours.

Récapitulatif — Points clés à retenir

1. ADCON1 = 0x0F → Toujours mettre en mode NUMÉRIQUE !
2. Dans l'ISR : effacer le FLAG d'interruption EN PREMIER (sinon boucle infinie).
3. Toute variable partagée entre l'ISR et main() doit être déclarée **volatile**.
4. Ordre d'activation des interruptions : INT0ED → GIE → PEIE → INT0IE → effacer INT0IF.
5. L'interruption est plus fiable que le polling pour les événements rapides ou courts.

Schéma de connexion

```
PIC18F4525
RB0 ---[R 10k vers VCC]---[Bouton S3 / S2]--- GND   (INT0 ou polling)
RA4 ---[R 10k vers VCC]---[Bouton polling]---- GND   (Scrutation Ex3)
RA4 ---[R 470 ohm]-----[LED D2 anode]----- GND   (Ex1)
RC2 ---[Buzzer piezoelectrique]-----GND   (Ex2 et Ex3)
```

Resistance pull-up : 10 kOhm vers +5V sur chaque bouton
Resistance LED : 470 Ohm en serie avec la LED